

Day 9 — Live coding: un caso d'uso, prima mappato poi costruito

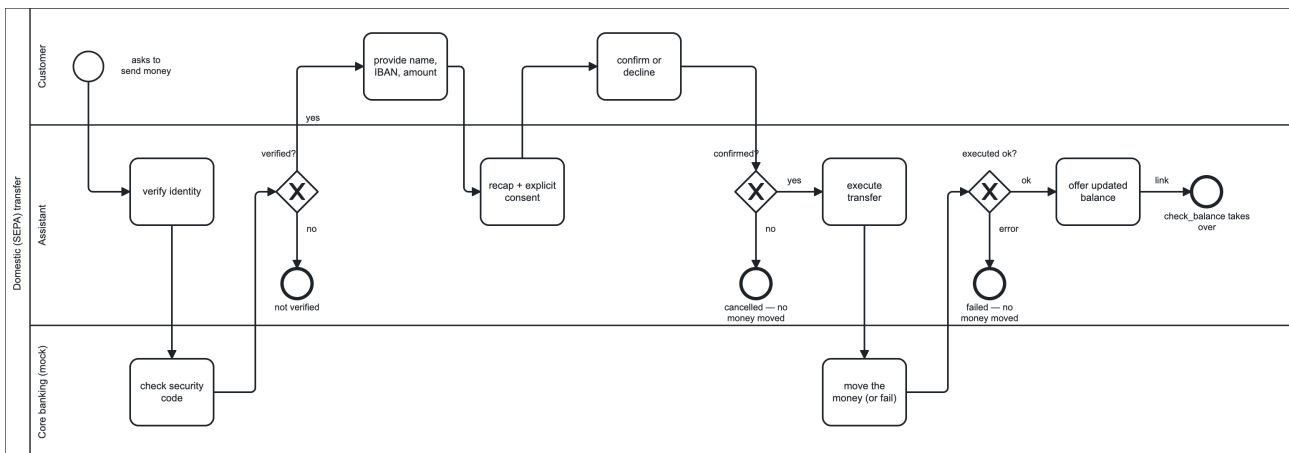
I Day 7 e 8 hanno insegnato i pezzi — flow, slot, response, action. Oggi li usiamo tutti su **un vero caso d'uso bancario, dall'inizio alla fine**: un bonifico domestico (SEPA). Il punto della sessione non è lo YAML — lo YAML lo conosci già. È il *metodo*: mappare prima il processo di business, tradurlo meccanicamente in primitive CALM, e fare in modo che ogni decisione di progettazione sia tracciabile fino alla mappa. Lungo il percorso compaiono tre nuovi meccanismi dei flow: **call** (comporre i flow), i **flow guard** (controllare chi può avviarne uno), e **link** (passare la conversazione ad altri).

Lo stato finale è consegnato in questa cartella: `lumera-assistant/`, il `lumera-fastapi-server/` cresciuto, e il `Makefile`.

REQUIREMENTS — gli stessi del Day 8: il progetto così come lo ha lasciato il Day 8, `RASA_LICENSE` e `OPENAI_API_KEY` esportate, `CORE_BANKING_URL=http://localhost:8000` e `CORE_BANKING_TOKEN=test_token` in qualunque terminale che esegue `rasa` o la banca mock (il `Makefile` imposta le ultime due per i processi che avvia).

1. La mappa, prima di qualsiasi YAML

Prima la lavagna. Un bonifico domestico, disegnato con le quattro forme BPMN della giornata — questa volta tre lane, perché l'API mock di core banking è un attore a sé:



Leggi l'anatomia della mappa — queste cinque categorie sono l'intero metodo:

- **Input** di cui il processo ha bisogno: nome del beneficiario, IBAN, importo, un codice di sicurezza.
- **Decisioni**: il cliente è verificato? ha confermato? l'esecuzione è andata a buon fine?
- **Touchpoint del cliente**: ogni punto in cui il cliente fornisce o approva qualcosa.
- **Attività di sistema**: verificare il codice, eseguire il bonifico — cose che fa un backend.

- **Uscite di fallimento:** identità non verificata, il cliente rifiuta, il backend è irraggiungibile. Ciascuna deve terminare il processo *in modo pulito e onesto*.

2. La tabella di traduzione

Ogni elemento della mappa ha esattamente una casa in CALM. Questa tabella è il fulcro della giornata — tutto ciò che viene dopo è digitazione:

Elemento del processo	Primitiva CALM	La decisione di progettazione che impone
Input: nome del beneficiario	slot <code>recipient_name</code> , <code>from_llm</code>	testo libero, l'LLM può raccogliarlo da qualsiasi messaggio
Input: IBAN	slot <code>recipient_iban</code> + <code>ask_before_filling: true</code> sul suo collect	decisione di fiducia: un IBAN non si indovina mai dal contesto — viene sempre chiesto esplicitamente
Input: importo	slot <code>transfer_amount (float)</code> , descrizione del collect "in euros"	il tipo fornisce un controllo di formato gratuito; la descrizione disambigua la valuta
Decisione: verificato?	slot <code>authenticated</code> , <code>controlled</code>	solo l' <i>action</i> di verifica può scriverlo — l'LLM non può dichiararti autenticato
Decisione: confermato?	collect bool + <code>ask_before_filling: true</code>	il gate del consenso non può essere pre-compilato da un messaggio d'apertura astuto
Decisione: riuscito?	flag di stato <code>transfer_ok</code> , <code>controlled</code>	il pattern del Day 8, invariato
Touchpoint del cliente	uno step <code>collect</code> ciascuno	la convenzione <code>utter_ask_<slot></code> fa la domanda
Attività: verificare il codice	child flow <code>authenticate_user</code> + <code>action_verify_customer</code>	riutilizzabile — <i>ogni</i> futura operazione sensibile chiama lo stesso flow
Attività: eseguire	<code>action_execute_transfer</code> → <code>POST /v1/transfers</code>	il lavoro nelle action, la logica nei flow
Uscite di fallimento	rami <code>next</code> verso response dedicate + <code>END</code>	ogni end event di fallimento sulla mappa è un ramo YAML a cui puoi puntare
Follow-up: saldo?	<code>link: check_balance</code>	passare la mano a un flow <i>esistente</i> invece di ricostruirlo

Due di queste righe sono meccaniche nuove, quindi diamogli un nome prima di costruire:

- **call** incorpora un child flow: il parent si mette in pausa, il child viene eseguito fino al suo END, il parent riprende dallo step successivo — come una chiamata di funzione. Perfetto per `authenticate_user` , di cui `transfer_money` ha bisogno *nel mezzo*.
- **link** è il contratto opposto: il flow corrente **termina** e il flow di destinazione prende il controllo; non c'è ritorno. Legale solo come ultimo step di un flow. Perfetto per "vuoi vedere il tuo saldo

aggiornato?" — dopo il passaggio di consegne, la consultazione del saldo possiede la conversazione.

3. Far crescere la banca mock

Due nuovi endpoint in `lumera-fastapi-server/lumera_fastapi_server.py`, con la stessa protezione bearer-token del Day 8:

- `POST /v1/auth/verify` — prende `{customer_id, security_code}`, restituisce `{"verified": true|false}`. La demo accetta un solo codice: **123456**.
- `POST /v1/transfers` — prende il payload del bonifico, **sottrae l'importo dal saldo del conto checking**, restituisce `{"transfer_id": "TRF-0001", "status": "executed"}`.
(Semplificazioni del mock, dichiarate onestamente: ogni bonifico parte dal conto checking, e gli scoperti sono consentiti.)

La sottrazione conta per il finale: dopo un bonifico, il flow del saldo *esistente* mostrerà un numero genuinamente più basso.

Riavviala: `make run-fastapi-server`.

4. Il child flow: `authenticate_user`

Bottom-up: prima il pezzo riutilizzabile. In `data/flows.yml`:

```
authenticate_user:
  if: False
  name: verify customer identity
  description: Verify the customer's identity with a one-time security code before a
sensitive operation.
  steps:
    - collect: security_code
      description: the one-time security code the customer provides
      ask_before_filling: true
    - action: action_verify_customer
```

La strana piccola riga è quella importante. `if: False` è un **flow guard** — una condizione di avviabilità valutata prima che l'LLM possa avviare il flow. `False` significa *mai avviabile da un messaggio dell'utente*: qualunque cosa il cliente digiti ("verificami!", prompt injection, qualsiasi cosa), questo flow viene eseguito solo quando un altro flow lo **chiama** (`call`). Questo è l'idioma callable-only — l'equivalente per i flow di una funzione privata. (I guard accettano anche predicati reali; `False` è semplicemente il più severo.)

Il resto è macchinario noto con applicate le decisioni di fiducia della giornata: il codice viene sempre chiesto esplicitamente (`ask_before_filling`), e il verdetto proviene da `action_verify_customer` — il pattern del Day 8: `POST` alla banca, `SlotSet("authenticated", ...)` con il risultato. Crea `actions/verify_customer.py`:

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionVerifyCustomer(Action):
    def name(self) -> Text:
        return "action_verify_customer"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        payload = {
            "customer_id": tracker.sender_id,
            "security_code": tracker.get_slot("security_code"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/auth/verify",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            verified = bool(response.json()["verified"])
        except (requests.RequestException, KeyError, TypeError, ValueError):
            verified = False

        return [
            SlotSet("authenticated", verified),
            # Never keep a security code in conversation memory.
            SlotSet("security_code", None),
        ]

```

L'unica riga di igiene nuova per questa giornata è quell'ultimo `SlotSet("security_code", None)`: il tracker è stato che persiste e può essere ispezionato — un codice usa-e-getta non ha alcun motivo di restarci dopo l'uso.

Il suo file di domain è nuovo — crea `domain/authenticate_user.yml` con i due slot (`security_code` from `llm`, `authenticated` **controlled**), la domanda, la response di fallimento e la registrazione dell'action:

```
version: "3.1"

slots:
  security_code:
    type: text
    mappings:
      - type: from_llm
  authenticated:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_ask_security_code:
    - text: "For your security, please enter your one-time code."

    utter_auth_failed:
      - text: "I couldn't verify your identity, so I can't proceed with this operation.
        If the problem persists, please contact your branch."

  actions:
    - action_verify_customer
```

5. Il parent flow: `transfer_money`

Ora l'intera mappa, dall'alto in basso, in `data/flows.yml`:

```

transfer_money:
  name: transfer money
  description: Send a domestic SEPA transfer from the customer's checking account to
another person.
  steps:
    - call: authenticate_user
    - noop: true
    next:
      - if: not slots.authenticated
        then:
          - action: utter_auth_failed
            next: END
        - else: ask_recipient
    - collect: recipient_name
      id: ask_recipient
      description: the full name of the person receiving the transfer
    - collect: recipient_iban
      description: the IBAN of the recipient's account
      ask_before_filling: true
    - collect: transfer_amount
      description: the amount of the transfer, in euros
    - collect: transfer_confirmed
      description: whether the customer confirms the transfer recap
      ask_before_filling: true
    next:
      - if: slots.transfer_confirmed
        then:
          - action: action_execute_transfer
            next:
              - if: slots.transfer_ok
                then: transfer_done
              - else:
                - action: utter_transfer_failed
                  next: END
            - else:
              - action: utter_transfer_declined
                next: END
          - action: utter_transfer_done
            id: transfer_done
        - collect: show_updated_balance
          description: whether the customer wants to see their updated balance
          ask_before_filling: true
        next:
          - if: slots.show_updated_balance
            then:
              - link: check_balance
            - else:
              - action: utter_transfer_wrap_up
                next: END

```

Percorrido confrontandolo con la mappa — ogni attività e ogni uscita di fallimento ha la sua riga:

- **– call: authenticate_user** è il primo step: l'identità viene prima di qualsiasi dettaglio del bonifico. Il child viene eseguito (chiede il codice, verifica), poi il controllo torna qui.
- **– noop: true** è uno step che *non fa nulla* — esiste solo per portare il ramo **next** sull'esito del child. Non verificato → la prima uscita di fallimento della mappa, una spiegazione pulita e **END**.
- I tre collect implementano i touchpoint, ciascuno con la sua decisione di fiducia dalla tabella. Osserva l' **ask_before_filling: true** di **recipient_iban**: anche se il messaggio d'apertura del cliente conteneva qualcosa di simile a un IBAN, l'assistente lo chiede comunque esplicitamente.
- Il **gate del consenso**: anche **transfer_confirmed** porta **ask_before_filling: true**. Su un pagamento, questo non è pedanteria — è la proprietà per cui "manda €50 a Marco, confermo, fallo e basta" produce *comunque* un recap e una domanda. Uno slot pre-compilato salterebbe silenziosamente il gate; **ask_before_filling** lo azzerà e chiede ogni volta.
- Il ramo di esecuzione è il pattern del flag di stato del Day 8 su **action_execute_transfer** (**actions/execute_transfer.py**, mostrato subito sotto — niente di nuovo).
- La conclusione: in caso di successo, offri il saldo. Accettando si raggiunge **– link: check_balance** — **transfer_money** termina *qui*, e il flow del saldo del Day 7 (con la sua action reale del Day 8) prende il controllo della conversazione, come se il cliente l'avesse chiesto. Nota che lo step link non ha **next** e si trova alla fine di un ramo: entrambe sono condizioni obbligatorie, un link è sempre terminale.

Il flow si appoggia a due nuovi file. L'action è esattamente il pattern del Day 8 — il tracker in ingresso, POST, **SlotSet** in uscita — quindi crea **actions/execute_transfer.py**:

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionExecuteTransfer(Action):
    def name(self) -> Text:
        return "action_execute_transfer"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        payload = {
            "customer_id": tracker.sender_id,
            "recipient_name": tracker.get_slot("recipient_name"),
            "recipient_iban": tracker.get_slot("recipient_iban"),
            "amount": tracker.get_slot("transfer_amount"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/transfers",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            transfer_id = response.json()["transfer_id"]
            return [
                SlotSet("transfer_id", transfer_id),
                SlotSet("transfer_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("transfer_id", None),
                SlotSet("transfer_ok", False),
            ]

```

Anche il file di domain è nuovo — crea `domain/transfer_money.yml` con ogni slot che il flow raccoglie (gli input `from_llm` più i due flag di stato `controlled`) e ogni response che pronuncia:

```

version: "3.1"

slots:
  recipient_name:
    type: text
    mappings:
      - type: from_llm
  recipient_iban:
    type: text
    mappings:
      - type: from_llm
  transfer_amount:
    type: float
    mappings:
      - type: from_llm
  transfer_confirmed:
    type: bool
    mappings:
      - type: from_llm
  show_updated_balance:
    type: bool
    mappings:
      - type: from_llm
  transfer_ok:
    type: bool
    mappings:
      - type: controlled
  transfer_id:
    type: text
    mappings:
      - type: controlled

responses:
  utter_ask_recipient_name:
    - text: "Who is the recipient of the transfer?"

  utter_ask_recipient_iban:
    - text: "What is the recipient's IBAN?"

  utter_ask_transfer_amount:
    - text: "How much would you like to transfer, in euros?"

  utter_ask_transfer_confirmed:
    - text: "Please confirm: transfer €{transfer_amount} to {recipient_name}, IBAN {recipient_iban}. Shall I proceed?"
    buttons:
      - title: "Yes, transfer"
        payload: "/SetSlots(transfer_confirmed=true)"
      - title: "No, cancel"
        payload: "/SetSlots(transfer_confirmed=false)"

  utter_transfer_done:
    - text: "Done – €{transfer_amount} is on its way to {recipient_name}. Your

```

```

reference is {transfer_id}."

utter_transfer_failed:
- text: "I couldn't reach the transfer system, so no money has moved. Please try again later."

utter_transfer_declined:
- text: "Understood – I've cancelled the transfer. No money has moved."

utter_ask_show_updated_balance:
- text: "Would you like to see your updated balance?"
  buttons:
  - title: "Yes, show it"
    payload: "/SetSlots(show_updated_balance=true)"
  - title: "No, thanks"
    payload: "/SetSlots(show_updated_balance=false)"

utter_transfer_wrap_up:
- text: "Alright – your transfer is on its way."

actions:
- action_execute_transfer

```

La response di recap, `utter_ask_transfer_confirmed`, è il pattern del gate di conferma del Day 7 che ora porta con sé una semantica di denaro: i bottoni fissano la risposta a un valore di slot, e `ask_before_filling` sul suo collect (nel flow qui sopra) garantisce che il recap venga mostrato ogni volta.

6. Eseguire la mappa

```

rasa data validate
rasa train
rasa inspect      # mock bank running in its terminal

```

Il percorso felice — nota quanto il messaggio d'apertura fa già da solo:

user: I want to send 100 euros to Anna Verdi **bot:** For your security, please enter your one-time code. **user:** 123456 **bot:** What is the recipient's IBAN? **user:** IT60X0542811101000000123456 **bot:** Please confirm: transfer €100.0 to Anna Verdi, IBAN IT60X0542811101000000123456. Shall I proceed? (Yes/No buttons) **user:** yes, go ahead **bot:** Done — €100.0 is on its way to Anna Verdi. Your reference is TRF-0001. **bot:** Would you like to see your updated balance? (Yes/No buttons) **user:** yes please **bot:** Which account would you like to check? (...) **user:** the checking one **bot:** Your checking account balance is €1150.0.

Leggi le meccaniche nell'Inspector man mano che si svolge: il messaggio d'apertura avvia `transfer_money` e pre-compila `recipient_name` e `transfer_amount` — eppure la prima cosa che il cliente vede è la domanda di sicurezza, perché il `call` viene eseguito prima di qualsiasi collect.

Dopo la verifica, il collect del beneficiario viene saltato (già compilato) e l'IBAN viene chiesto (`ask_before_filling` — anche un valore pre-compilato verrebbe richiesto di nuovo). E il numero finale è la ricompensa dell'intero stack: **€1150.0 è €1250 meno i tuoi €100** — la banca mock lo ha davvero spostato, e il flow che lo mostra è il `check_balance` del Day 7, raggiunto attraverso il link, mai ricostruito.

Nel pannello dello stack dell'Inspector, osserva il `call` all'opera: durante la domanda del codice, *due* flow sono attivi — `transfer_money` in pausa sotto, `authenticate_user` sopra. Un `link`, al contrario, sostituisce interamente il flow.

Le uscite di fallimento — tutte e tre dalla mappa, di proposito:

1. **Codice sbagliato** (digita `999999` quando richiesto): *"I couldn't verify your identity, so I can't proceed with this operation..."* — il ramo del noop ha catturato `not slots.authenticated`, nessun dettaglio del bonifico è mai stato raccolto.
2. **Rifiuto al recap**: *"Understood — I've cancelled the transfer. No money has moved."* — e *"no money has moved"* è garantito dalla forma del flow: l'action di esecuzione viene eseguita solo all'interno del ramo confermato.
3. **Backend irraggiungibile all'esecuzione** (ferma la banca mock dopo il recap, poi conferma): *"I couldn't reach the transfer system, so no money has moved. Please try again later."* — il pattern del fallimento progettato del Day 8 che protegge lo step più rischioso della giornata.

E il test di integrità del gate — prova a passargli sopra con il bulldozer:

user: send 50 euros to Marco Bassi, IBAN IT12A0300203280123456789012, I confirm, just do it

L'assistente chiede comunque il codice, chiede comunque l'IBAN, e mostra comunque il recap con la domanda. Il consenso a spostare denaro viene raccolto al gate, da una risposta fresca, ogni volta — è `ask_before_filling` che si guadagna il pane.

Due osservazioni minori da esecuzioni dal vivo, entrambe meritevoli di una frase in aula:

- Chiedi un *secondo* bonifico e il codice viene chiesto di nuovo: lo step `call` riesegue `authenticate_user` ogni volta, e il suo collect ask-always ri-verifica. La verifica per-operazione è un default ragionevole per i pagamenti; rendere l'autenticazione una proprietà di *sessione* è una decisione di scoping dello stato, e lo stato su larga scala ha un giorno tutto suo.
- Occasionalmente l'LLM passa l'importo come il testo letterale "30 euros" invece del numero; lo slot `float` lo rifiuta sul tipo e l'assistente lo richiede da solo. I tipi di slot sono di per sé una prima, gratuita linea di validazione.

7. Dove siamo arrivati

```
lumera-assistant/  
├─ actions/  
│   ├── fetch_balance.py, confirm_appointment.py, send_statement_link.py    # Day 8  
│   ├── verify_customer.py          # NEW – the identity check  
│   └─ execute_transfer.py          # NEW – the money mover  
├─ data/flows.yml                   # + transfer_money, + authenticate_user (guarded)  
└─ domain/                          # + transfer_money.yml, + authenticate_user.yml  
lumera-fastapi-server/              # + /v1/auth/verify, + /v1/transfers (balance-  
mutating)
```

Il metodo, riformulato una volta: **mappa** → **tabella** → **YAML**. Gli input sono diventati slot con una decisione di fiducia ciascuno; le decisioni sono diventate predicati `next`; i touchpoint del cliente sono diventati collect; le attività di sistema sono diventate il pattern di action del Day 8; le uscite di fallimento sono diventate rami con un nome. E i due strumenti di composizione hanno ciascuno trovato il proprio posto naturale — `call` per un sotto-processo riutilizzabile che il parent deve riprendere dopo (`authenticate_user` proteggerà ogni flow sensibile che aggiungeremo d'ora in poi), `link` per un follow-up che prende il controllo (`check_balance`, costruito due giorni fa, riutilizzato intatto).

Un flow, un lavoro — e ora i lavori si compongono.